B.Tech. BCSE497J - Project-I

AI-BASED MUSIC INSTRUMENT RECOGNITION SYSTEM

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

23BCE0775 **ATHARV SWARNKAR**

Under the Supervision of

RAJARAJAN G
Assistant Professor Sr. Grade 1
School of Computer Science and Engineering (SCOPE)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

I hereby declare that the project entitled "AI-BASED MUSIC INSTRUMENT RECOGNITION SYSTEM" submitted by me, for the award of the degree of Bachelor of Technology in Computer Science and Engineering to VIT is a record of project work carried out by me under the supervision of RAJARAJAN G, Assistant Professor Sr. Grade 1.

I further declare that the work reported in this project has not been submitted, either in part or in full, for the award of any other degree or diploma at this institute or any other institute or university.

Place : Vellore
Date : 27th October 2026

Signature of the Candidate
ATHARV SWARNKAR (23BCE0775)

CERTIFICATE

This is to certify that the project entitled "AI-BASED MUSIC INSTRUMENT RECOGNITION SYSTEM" submitted by ATHARV SWARNKAR (23BCE0775), School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of project work carried out by the student under my supervision during Fall Semester 2026–2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted, either in part or in full, for the award of any other degree or diploma at this institute or any other institute or university. The project report is submitted for academic evaluation in accordance with the university requirements.

Place : Vellore

Date : October 2026

Signature of the Guide
RAJARAJAN G
Assistant Professor Sr. Grade 1

Examiner(s)

Dr. BOOMINATHAN P

Computer Science and Engineering

ACKNOWLEDGEMENTS

I am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired me to strive for excellence. The Dean's dedication to academic excellence and innovation has been a constant source of motivation for me. I appreciate his efforts in creating an environment that nurtures creativity and critical thinking.

I express my appreciation to the faculty and staff of the School of Computer Science and Engineering (SCOPE) for their academic support and for providing an environment conducive to project development.

I am grateful to my project guide, RAJARAJAN G, Assistant Professor Sr. Grade 1, for guidance and feedback during the development of this project.

Thank you all for your contributions and support.

ATHARV SWARNKAR

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	
1.	**INTRODUCTION**	
	1.1 Background	
	1.2 Motivation	
	1.3 Scope of the Project	
2.	**PROJECT DESCRIPTION AND GOALS**	
	2.1 Literature Review	
	2.2 Research Gap	
	2.3 Objectives	
	2.4 Problem Statement	
	2.5 Project Plan	
3.	**TECHNICAL SPECIFICATION**	
	3.1 Requirements	
	3.2 Feasibility Study	
	3.3 System Specification	
4.	**DESIGN APPROACH AND DETAILS**	
	4.1 System Architecture	
	4.2 Design and Diagrams	
	4.3 Data Representation and Dataset	
5.	**METHODOLOGY AND TESTING**	
	5.1 Methodology	
	5.2 Testing Strategy	
6.	**PROJECT DEMONSTRATION**	
7.	**RESULT AND DISCUSSION**	
8.	**CONCLUSION**	
9.	**REFERENCES**	
	APPENDIX A – SAMPLE CODE	

List of Figures

Figure No.	Title	Page No.
4.1	System architecture and analysis flow	
4.2	Data flow from upload to instrument analysis	

Figure 4.1 System architecture and analysis flow

Figure 4.2 Audio processing sequence

Figure 5.1 Validation and testing workflow

(In the chapters, figure caption should come below the figure and table caption should come above the table. Figure and table captions should be of font size 10.)

List of Tables

Table No. Title Page No.

Table 3.1 Functional requirements

List of Abbreviations

API Application Programming Interface

CLAP Contrastive Language-Audio Pretraining

CPU Central Processing Unit

GPU Graphics Processing Unit

HTTP Hypertext Transfer Protocol

MIR Music Information Retrieval

RMS Root Mean Square

UI User Interface

ABSTRACT

Music contains multiple overlapping sources, including instruments, percussion, vocals, and production effects. Identifying the instruments that contribute to a mixed recording is challenging because their sounds overlap in time and frequency, their loudness varies across a song, and the same instrument can sound different across genres and recording conditions. This project, titled "AI-Based Music Instrument Recognition System," develops a web-based workflow for analysing an uploaded audio file and estimating which instrument categories are represented in its sound. The implementation combines audio preprocessing, source separation, pretrained audio-language matching, window-level analysis, and a browser-based results interface.

The backend is implemented using FastAPI. It accepts MP3, WAV, and FLAC files, checks the extension and upload size, saves the input under a generated identifier, and verifies that the audio can be decoded. Demucs is invoked for source separation using the htdemucs model. The current analysis path selects the separated "other" stem for further instrument matching. The Contrastive Language-Audio Pretraining (CLAP) model, loaded through the Hugging Face Transformers interface, compares short audio windows with text prompts describing instrument categories. The audio is converted to mono at 48 kHz and divided into one-second windows, with the final partial window padded to the required size. Windows are processed in batches to limit per-call overhead.

The system aggregates the window-level match scores into song-level instrument match indices and also retains timeline data for visualisation. Scores are bounded to the interval from zero to one before aggregation and are reported on a 0–100 scale. These values are similarity-derived indices; they are not calibrated probabilities, measured loudness shares, or verified physical contributions to the original mix. The React frontend provides audio selection, local playback, waveform display, analysis status, ranked instrument summaries, a time-based view, and JSON-compatible result handling. The frontend communicates with the backend through the '/api/analyze' endpoint.

The implementation also includes additional modules for refining instrument analysis and score interpretation. The instrument detector uses expanded descriptive prompts for multiple instrument categories and adjusts CLAP similarity scores against a generic-music baseline and a minimum evidence threshold. The project further incorporates a stem-analysis module that estimates the relative RMS energy of separated stems, a top-k softmax-based percentage calculator, and a final-percentage module that combines stem-distribution values with instrument estimates. In addition, an independent Explainable Music Information Retrieval (MIR) style-analysis module has been developed to extend the system beyond instrument recognition. This module extracts a unified feature representation of an audio recording using characteristics such as tempo, estimated musical key and mode, harmonic and percussive energy, spectral centroid, spectral roll-off, spectral bandwidth, spectral flat-

ness, zero-crossing rate, Mel-frequency cepstral coefficients (MFCCs), root mean square (RMS) energy, and dynamic range. It also derives heuristic genre and mood labels, produces instrument-related spectral-evidence indices, and supports similarity-based song retrieval through weighted feature comparisons. Retrieved song metadata is subsequently used to infer artist-style similarity, while the explanation mechanism communicates the evidence and limitations underlying the results. The MIR module is implemented separately from the primary instrument-analysis backend and uses documented heuristic rules for genre, mood, and instrument-related labels; these should not be interpreted as outputs of trained or calibrated classifiers. Similarly, the similarity index represents a ranking measure rather than a probability, and artist-style inference depends on the availability of sufficient matching catalogue evidence. The primary instrument-analysis endpoint continues to return mean window-level CLAP match indices and window-coverage statistics, while the MIR module provides a complementary feature-based analysis and retrieval workflow. Automated test files are included for both analysis components, although their presence does not independently establish that all tests have passed. Together, these components establish a modular foundation for explainable music analysis, combining instrument recognition, acoustic feature extraction, similarity-based retrieval, and evidence-based artist-style inference, with further integration, empirical evaluation, and calibration required before making quantitative performance claims.

CHAPTER 1

1. INTRODUCTION

1.1 Background

Music Information Retrieval (MIR) is an interdisciplinary field that applies computational techniques to analyse, organise, and interpret musical audio. It encompasses tasks such as instrument recognition, tempo estimation, musical key detection, genre classification, mood analysis, and music similarity assessment. A recorded song typically consists of multiple overlapping sound sources, including guitars, bass, drums, keyboards, strings, brass, woodwinds, vocals, and electronic elements. Identifying and characterising these components automatically is challenging because instruments frequently overlap in time and frequency, their acoustic properties vary across recording conditions, and their prominence changes throughout a musical composition.

Recent advances in machine learning have enabled audio analysis systems to learn meaningful representations of sound and compare audio signals with natural-language descriptions. Pretrained audio-language models, such as Contrastive Language-Audio Pretraining (CLAP), support similarity-based matching between audio segments and descriptive text prompts. In parallel, source-separation models such as Demucs can estimate individual stems from a mixed recording, providing an alternative representation for subsequent analysis. These techniques offer a practical foundation for developing instrument-recognition systems without requiring the training of a large audio model from scratch.

Beyond identifying individual instruments, a broader MIR-based approach can represent a song through multiple acoustic and musical characteristics. Features such as tempo, estimated musical key, spectral properties, harmonic and percussive content, and Mel-frequency cepstral coefficients (MFCCs) can be combined to compare recordings and investigate similarities in musical style. Such representations can support feature-based song retrieval and provide interpretable evidence for analysing relationships between recordings. This project therefore considers both instrument recognition and a complementary MIR-style analysis workflow, with the aim of making music analysis more informative and accessible through a web-based application.

1.2 Motivation

The primary motivation of this project is to develop an accessible system that analyses an uploaded music recording and presents meaningful information about its instrumental content. Manual identification of instruments and their changing presence throughout a song can require musical expertise and considerable listening effort. An automated system can assist this process by examining audio segments, estimating instrument-related similarity scores, and presenting the resulting information through a visual interface.

A further motivation is to extend analysis beyond isolated instrument labels. Different recordings may share similarities in tempo, harmonic structure, timbral characteristics, or broader

musical style even when their instrumentation differs. A feature-based MIR workflow can use these characteristics to represent recordings, retrieve acoustically similar songs, and provide evidence supporting the resulting comparisons. Combining these capabilities creates a more comprehensive framework for exploring musical content than instrument recognition alone.

The project also emphasises modularity and interpretability. Separating audio validation, source separation, feature extraction, model inference, score aggregation, similarity computation, and result presentation into distinct components makes the system easier to maintain and extend. A browser-based interface further allows users to interact with the analysis pipeline without directly operating its underlying processing modules.

1.3 Scope of the Project

The project comprises two complementary areas of music analysis: instrument recognition and MIR-style feature analysis. The instrument-recognition pipeline accepts supported audio files, validates the uploaded data, performs source separation using Demucs, and applies CLAP-based text–audio similarity matching to one-second audio windows. The resulting window-level scores are aggregated into song-level instrument match indices and time-based coverage information. A FastAPI backend coordinates the processing pipeline, while a React frontend supports audio selection, playback, waveform visualisation, analysis status, and presentation of the results.

The MIR-style analysis component extends the scope of the project through acoustic feature extraction and feature-based comparison of musical recordings. Its analysis includes characteristics such as tempo, estimated musical key and mode, spectral descriptors, MFCCs, RMS energy, and harmonic and percussive information. These features support the construction of a unified representation for song similarity and retrieval. The broader workflow also incorporates heuristic genre and mood categorisation, instrument-related spectral evidence, and artist-style similarity inference based on retrieved recordings. Explanations are intended to communicate the features and comparisons underlying these outputs rather than present them as unsupported model certainty.

The two analysis areas are modular and should be distinguished when interpreting their outputs. Instrument-level CLAP scores are similarity-derived indices rather than calibrated probabilities or direct measurements of an instrument's physical contribution to a recording. Likewise, heuristic genre and mood labels, feature-based similarity scores, and artist-style inferences require appropriate evaluation before they can be treated as definitive classifications. The scope of this project includes the implemented processing workflows and their associated limitations, while further integration, calibration, and systematic evaluation remain necessary for establishing reliable performance across diverse musical recordings.

1.4 Organisation of the Report

This report is organised into nine chapters, followed by an appendix containing representative implementation code. Chapter 1 introduces the background, motivation, and scope of the project. Chapter 2 presents the project objectives, literature review, research gap, problem statement, and development plan. Chapter 3 describes the functional and non-functional requirements, feasibility considerations, and hardware and software specifications. Chapter 4 explains the system architecture, design diagrams, data representation, and dataset considerations. Chapter 5 details the methodology used for audio processing, instrument recognition, MIR-style feature analysis, similarity computation, and testing. Chapter 6 describes the system demonstration workflow and its current implementation boundaries. Chapter 7 discusses the findings supported by the available implementation, the interpretation of analysis scores, limitations, and potential enhancements. Chapter 8 concludes the report and identifies directions for future development. Chapter 9 presents the references, and Appendix A provides representative code and descriptions of selected implementation components.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Music Information Retrieval (MIR) involves computational techniques for extracting, analysing, and interpreting musical information from audio recordings. The field encompasses tasks such as instrument recognition, tempo estimation, musical key detection, genre and mood analysis, and music similarity assessment. These tasks are challenging because musical recordings contain overlapping sound sources whose acoustic characteristics vary with instrumentation, arrangement, recording conditions, and production effects.

Demucs is a deep-learning-based music source-separation system that estimates constituent source stems from a mixed recording. In this project, the htdemucs model is invoked as an external command-line process, and the resulting `other.wav` stem is selected for subsequent instrument analysis. Source separation provides an intermediate representation that may expose instrument-related evidence more clearly. However, the estimated stems are not perfect isolations and may retain interference or omit relevant information present in other stems.

Contrastive Language-Audio Pretraining (CLAP) learns a shared representation of audio and natural-language descriptions. This enables audio segments to be compared with text prompts describing candidate sound categories, supporting zero-shot similarity-based instrument matching. The implementation applies CLAP to one-second audio windows using descriptive prompts for instrument categories such as guitar, drums and percussion, piano and keyboards, strings, brass, woodwinds, and bass. The resulting window-level scores are aggregated into song-level match indices, while temporal information is retained to represent changes in instrument-related evidence throughout a recording. These scores are similarity-derived indices, not calibrated probabilities of instrument presence or direct measurements of an instrument's contribution to the original mix.

A complementary approach to music analysis is feature-based MIR, which characterises recordings using acoustic and musical descriptors. Features such as spectral centroid, spectral roll-off, spectral bandwidth, spectral flatness, zero-crossing rate, root mean square (RMS) energy, and Mel-frequency cepstral coefficients (MFCCs) describe different aspects of a signal's spectral and timbral characteristics. Harmonic and percussive analysis, together with tempo and estimated musical key and mode, provides additional information about rhythmic and tonal structure. A combined feature representation can support comparisons between recordings and provide a basis for similarity ranking and song retrieval.

The MIR-style analysis component extends the project beyond instrument recognition by incorporating acoustic feature extraction, musical attribute analysis, feature-based song similarity, and retrieval-based artist-style inference. It also considers genre and mood categorisation and the presentation of explanations for analytical results. These outputs must be interpreted according to their underlying methods: heuristic labels are not equivalent

to validated classifier predictions, and similarity rankings do not independently establish musical genre or artist identity. The feature-based workflow therefore complements the CLAP-based instrument pipeline by examining broader musical characteristics rather than relying exclusively on instrument-related similarity scores.

The application architecture uses FastAPI for backend request handling, Librosa for audio loading and preprocessing, and React for the user interface. These components support audio submission, processing status, playback, waveform visualisation, and presentation of analysis results. The two analytical workflows have distinct responsibilities and should be evaluated separately, even when their outputs are presented through a common interface.

2.2 Research Gap

Automated music analysis presents several interrelated challenges. Instrument recognition in a mixed recording is affected by source overlap, imperfect source separation, and variations in the acoustic characteristics of instruments across recordings. Although audio–text similarity models provide a flexible method for identifying candidate instrument categories, their scores depend on the audio representation and descriptive prompts used. Furthermore, a single recording-level score may conceal temporal variations in the evidence supporting an instrument label.

Feature-based music analysis addresses a different requirement by representing recordings through measurable acoustic and musical characteristics. However, extracting features alone does not guarantee meaningful song similarity, reliable retrieval, or valid artist-style inference. The choice of features, their relative weighting, the similarity measure, and the availability of suitable reference recordings can all affect the resulting rankings. Genre and mood estimates also require validation before they can be treated as reliable classifications.

The research gap addressed by this project is the need for a modular music-analysis system that combines instrument-level evidence with broader musical feature analysis while preserving the distinction between their outputs. The instrument-recognition workflow uses source separation and CLAP-based matching to estimate instrument similarity across short audio windows. The MIR-style workflow uses acoustic and musical descriptors to characterise recordings and support feature-based comparison, retrieval, and artist-style analysis. Together, these approaches provide complementary perspectives on a recording and support a more comprehensive analysis than either approach alone.

A further requirement is to make the resulting analysis interpretable. The system must communicate how instrument scores are calculated, retain relevant temporal or feature-level information, and distinguish model-derived indices from calibrated predictions. It must also provide a structured basis for evaluating processing reliability, retrieval relevance, robustness, and computational performance. These considerations motivate the project's modular architecture and the need for systematic testing before quantitative performance claims can be made.

2.3 Objectives

The primary objective of this project is to develop a web-based music analysis system combining instrument recognition with feature-based Music Information Retrieval. The specific objectives are:

- Develop an audio-upload and validation pipeline that accepts supported audio formats and handles empty, oversized, unsupported, or undecodable files appropriately.
- Apply Demucs-based source separation and CLAP-based audio–text similarity matching to identify candidate instrument categories within short audio windows.
- Aggregate window-level instrument scores into song-level match indices and generate temporal information for examining changes in instrument-related evidence throughout a recording.
- Extract acoustic and musical features, including spectral descriptors, MFCCs, RMS energy, harmonic and percussive information, tempo, and estimated tonal attributes, to construct a representation of a recording.
- Develop feature-based song similarity and retrieval capabilities that rank recordings according to their musical and acoustic characteristics.
- Incorporate genre and mood categorisation, instrument- related spectral evidence, and retrieval-based artist-style inference, while distinguishing heuristic outputs from validated predictions.
- Provide interpretable results through a web interface that presents instrument scores, temporal evidence, musical features, similarity rankings, and relevant explanations.
- Maintain a modular architecture using FastAPI and React so that the instrument-recognition and MIR-style components can be developed, integrated, and evaluated independently.
- Establish a verification and evaluation framework covering functional correctness, precision, recall, F1-score, retrieval relevance, latency, and robustness, with quantitative conclusions based on reproducible experimental evidence.

2.4 Problem Statement

Given a music recording, the system must derive complementary information about its instrumental content and broader musical characteristics. The first task is to estimate which instrument categories are supported by the audio evidence and represent how the associated similarity scores vary over time. The second task is to characterise the recording through acoustic and musical features that can support similarity-based comparison, retrieval of related songs, and artist-style analysis.

The problem is challenging because a musical mixture contains overlapping sources, acoustic properties change throughout a recording, and different analytical methods produce outputs with different interpretations. CLAP similarity scores do not directly represent calibrated probabilities of instrument presence, while feature-based similarity scores indicate relative resemblance under a selected representation and comparison method. Similarly, heuristic genre, mood, and artist-style outputs require appropriate validation before they can be treated as definitive conclusions.

The objective is therefore to establish a modular and accessible music-analysis workflow that combines instrument recognition with MIR-style feature analysis, supports meaningful presentation of results, and preserves the distinction between measured features, similarity-derived indices, and higher-level inferences. The system should provide a foundation for further evaluation and extension without overstating the reliability of outputs that have not yet been experimentally validated.

2.5 Development Methodology and Project Plan

The project follows a modular development approach in which the instrument-recognition and MIR-style analysis workflows are developed as distinct but complementary components. The instrument-recognition pipeline focuses on audio validation, source separation, window-level CLAP matching, and score aggregation. The MIR-style workflow focuses on acoustic feature extraction, musical attribute analysis, feature-based similarity, retrieval, and interpretation.

The development process begins with problem formulation and requirements analysis, followed by the implementation of the individual processing components. The analytical workflows are then connected to the application interface, enabling users to submit audio and inspect the resulting information. Verification activities cover individual components as well as the complete user workflow. Quantitative evaluation is treated separately from implementation and requires suitable test data, defined metrics, and reproducible measurements.

The principal development stages and their intended outputs are summarised in Table 2.1.

Stage	Development activity	Intended output
1	Problem formulation and requirements analysis	Defined project scope, technical requirements, analytical objectives, and relevant literature.
2	Instrument-recognition pipeline	Audio validation, Demucs-based source separation, CLAP window matching, song-level instrument indices, and temporal evidence.
3	MIR feature analysis and retrieval	Acoustic feature representation, musical attribute analysis, feature-based similarity ranking, and retrieval-related outputs.
4	Interface development and integration	Backend communication, audio playback, waveform visualisation, processing feedback, and structured result presentation.
5	Verification and evaluation	Component-level and workflow testing, followed by evaluation of prediction quality, retrieval relevance, robustness, and latency where supported by experimental data.

The stages describe the development structure and intended outputs; they do not independently establish that every component has been completed or experimentally validated. Implementation status must be determined from the relevant code and available test evidence. Likewise, conclusions about recognition quality, retrieval effectiveness, and runtime performance require reproducible measurements. This distinction provides a consistent basis for documenting the current system and identifying the work required for further development and evaluation.

CHAPTER 3

3. TECHNICAL SPECIFICATION

3.1 Requirements

The technical requirements define the expected capabilities, implementation constraints, and operational characteristics of the proposed music-analysis system. The specification covers two complementary analytical workflows: an instrument-recognition pipeline based on source separation and audio–text similarity, and a feature-based Music Information Retrieval (MIR) pipeline for musical attribute analysis and similarity-based retrieval. The implementation status of each requirement must be interpreted in relation to the relevant source modules; the existence of a component does not, by itself, establish its integration, accuracy, or successful execution.

3.1.1 Functional Requirements

The functional requirements describe the operations that the application is expected to perform, from audio submission and preprocessing to analysis and presentation. Table 3.1 summarises the requirements and their implementation status based on the project details documented in this report.

ID	Requirement	Implementation status
FR-01	Accept audio files through the web interface.	Implemented in the React frontend.
FR-02	Validate supported MP3, WAV, and FLAC formats and enforce a maximum upload size of 100 MB.	Implemented through backend upload validation.
FR-03	Decode the submitted recording, convert it to the required audio representation, and determine its duration.	Implemented using Librosa-based audio preprocessing.
FR-04	Apply source separation to obtain audio stems for downstream analysis.	Demucs with the `htdemucs` model is invoked by the instrument-analysis workflow.
FR-05	Estimate instrument-category matches from short audio windows using audio–text similarity.	Implemented using CLAP and descriptive text prompts over one-second windows.
FR-06	Aggregate window-level instrument scores and retain temporal analysis information.	Implemented in the instrument-analysis service and analysis engine.

ID	Requirement	Implementation status
FR-07	Present the recording and its analysis through playback, waveform visualisation, and instrument summaries.	Implemented in the React interface, according to the documented frontend functionality.
FR-08	Extract acoustic descriptors, including spectral features, MFCCs, and energy-related characteristics.	Provided by the complementary feature-based MIR workflow.
FR-09	Estimate musical attributes such as tempo and musical key and mode.	Included in the MIR-style analysis workflow; estimation accuracy is not established by the available implementation description.
FR-10	Produce genre and mood labels.	Provided through heuristic analysis in the MIR-style workflow; not established as a trained or calibrated classification system.
FR-11	Compare recordings and retrieve similar songs using feature-based similarity.	Included in the MIR-style retrieval workflow. Retrieval quality requires evaluation against suitable reference data.
FR-12	Infer artist-style similarity from retrieved recordings and associated metadata.	Included as a retrieval-based analysis function; its reliability depends on the available catalogue and metadata.
FR-13	Provide explanations that communicate the evidence and limitations associated with analysis results.	Included in the MIR-style analysis design; the scope of explanation and its empirical usefulness require evaluation.
FR-14	Expose health and analysis endpoints for application communication.	The documented instrument-analysis backend provides `GET /`, `GET /api/health`, and `POST /api/analyze`. The MIR workflow should be evaluated separately where it is not connected to these endpoints.

The requirements distinguish the principal instrument-analysis endpoint from the complementary MIR workflow. In particular, the presence of tempo estimation, genre or mood labelling, similarity retrieval, and artist-style inference in the project does not imply that these functions are all returned by the primary instrument-analysis API. Their implementation and integration must be assessed at the level of the relevant modules.

The instrument scores are similarity-derived indices calculated from audio–text matching

and subsequent aggregation. They should not be interpreted as calibrated probabilities, verified instrument-isolation measurements, or direct percentages of an instrument's contribution to the original recording. Similarly, feature-based similarity values represent relative resemblance under the selected feature representation and comparison method.

3.1.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints that influence the reliability, maintainability, usability, and deployment of the application.

- **Reliability:** The application should handle unsupported formats, invalid uploads, decoding failures, and processing errors through controlled error responses. End-to-end reliability must be established through repeatable testing.

- **Maintainability:** Audio preprocessing, model loading, source separation, score aggregation, API handling, and user-interface functionality should remain organised into distinct modules to support debugging and independent development.

- **Performance:** Audio-window processing should use batching where appropriate, and expensive models should be reused rather than unnecessarily reloaded for every request. Actual latency depends on the recording duration, hardware, source-separation overhead, and model-inference configuration.

- **Usability:** The interface should provide clear upload controls, processing feedback, error messages, audio playback, and structured presentation of analysis results.

- **Interpretability:** Results should identify whether they represent extracted measurements, similarity-derived scores, heuristic labels, or retrieval-based inferences. The interface and report must not present unvalidated estimates as definitive predictions.

- **Scalability:** The application should support future separation of computationally intensive tasks from request handling. Processing a large catalogue requires dedicated ingestion, storage, scheduling, and monitoring mechanisms beyond a single interactive web application.

- **Security:** Upload validation and file-size restrictions provide initial safeguards. A production deployment should additionally apply appropriate authentication and authorisation, rate limiting, secure temporary-file handling, restricted CORS policies, dependency management, and resource limits.

- **Compatibility:** The frontend, backend, audio libraries, and pretrained models must operate within a compatible software environment. Model availability and dependency versions should be documented for reproducibility.

- **Data governance:** Audio files and reference catalogues must be processed

in accordance with applicable copyright, licensing, privacy, and distribution requirements.

These requirements establish the intended quality attributes of the system. They should not be interpreted as evidence that every attribute has already been verified under production conditions.

3.2 Feasibility Study

The feasibility study examines whether the proposed analytical workflows can be developed and operated using the available software technologies and resources. It considers technical feasibility, economic considerations, and the practical implications of making automated music analysis accessible to users.

3.2.1 Technical Feasibility

The system uses established Python and JavaScript technologies for audio processing, model inference, backend request handling, and browser-based interaction. Librosa supports audio decoding and preprocessing, while Demucs provides pretrained source-separation functionality. CLAP enables comparison between audio segments and descriptive text prompts, providing a flexible approach to instrument-category matching without requiring a separately trained classifier for every category.

The complementary MIR workflow uses acoustic and musical descriptors to represent recordings and support feature-based comparison. This approach can be implemented using established signal-processing methods and numerical feature representations. However, the usefulness of similarity retrieval, genre and mood estimates, and artist-style inference depends on feature selection, weighting, reference-catalogue quality, and evaluation methodology.

The system is technically feasible as a modular music-analysis application. Its computational requirements are nevertheless non-trivial: source separation and neural-model inference may impose substantial CPU, GPU, and memory demands. Performance will vary with the duration of the input, the available hardware, and the software configuration. Processing millions of audio files would require a dedicated data-ingestion and storage architecture and cannot be inferred from the existence of an interactive upload-and-analysis application.

3.2.2 Economic Feasibility

The core implementation is based primarily on open-source software and pretrained models. This reduces the need to develop every analytical component from the beginning, although it does not eliminate infrastructure and maintenance costs. The principal cost factors include compute resources, storage for recordings and intermediate source stems, network transfer, model downloads, deployment infrastructure, and developer effort.

The economic feasibility of a larger deployment depends on the expected number and duration of recordings, processing throughput, retention requirements, and hardware utilisation.

A reliable numerical cost estimate would require measured resource consumption and a defined deployment configuration. Since these measurements and a verified infrastructure budget are not available in the documented project details, no numerical cost estimate is asserted here.

3.2.3 Social Feasibility

The application may be useful to students, musicians, educators, and researchers who wish to explore instrumental content and broader acoustic characteristics of music. Combining instrument-related evidence with feature-based retrieval may help users examine recordings from different analytical perspectives.

The outputs must, however, be presented with appropriate qualifications. Similarity scores and heuristic genre or mood labels can be affected by overlapping instruments, recording quality, production effects, and the characteristics of the underlying models or features. The system should therefore support exploration and comparison rather than imply that all results constitute definitive musical judgments.

Any audio collection used for analysis or retrieval must also be managed according to its applicable licences and copyright restrictions. If the system is extended to a large catalogue, data provenance, permitted uses, storage policies, and access controls should be documented.

3.3 System Specification

The system specification describes the hardware and software environment required to execute the analytical workflows. Exact minimum requirements depend on the models, audio duration, concurrency, and deployment configuration. Accordingly, the following specification identifies the relevant resource categories and the technologies documented for the project without asserting unverified hardware benchmarks.

3.3.1 Hardware Specification

The application requires a computer capable of running the backend service, decoding audio, loading the necessary models, and storing temporary processing outputs. The principal hardware considerations are:

- **Processor:** A modern multi-core CPU is required for backend execution, audio preprocessing, file operations, and tasks that are not accelerated by a GPU.
- **Memory:** Sufficient RAM is required to hold the application, audio buffers, model components, and intermediate processing data. Requirements increase with model size, recording duration, and concurrent requests.
- **GPU:** A compatible GPU may accelerate supported neural-model operations when the installed PyTorch and model configuration permit GPU execution. The system should also account for CPU execution or partial CPU offloading where GPU memory is insufficient.

• **Storage:** Disk space is required for model weights, uploaded recordings, temporary separated stems, application dependencies, and any reference catalogue used by the MIR workflow.

• **Network:** Network access may be required to download pretrained models and dependencies during initial setup. A deployed application also requires connectivity between the browser interface and backend service.

The exact CPU model, GPU model and memory, installed RAM, and available storage should be recorded from the machine used for the final demonstration. These values must be verified directly rather than inferred from the application's software dependencies.

3.3.2 Software Specification

The software stack consists of a browser-based interface, a Python backend, audio-processing libraries, pretrained neural models, and HTTP-based communication. Table 3.2 summarises the documented technologies and their roles.

Layer	Technology	Purpose
Frontend	React 18, Vite, JavaScript, Lucide React, and Recharts dependencies	Provides the browser interface and supports user interaction and visual presentation.
Backend	Python and FastAPI	Handles HTTP requests and orchestrates the documented instrument-analysis workflow.
Audio preprocessing	Librosa and NumPy	Supports audio decoding, mono conversion, resampling, duration handling, and generation of analysis windows.
Source separation	Demucs with the `htdemucs` model	Estimates source stems from the mixed recording for downstream instrument analysis.
Audio–text similarity	Hugging Face Transformers and `laion/clap-htsat-unfused`	Compares audio windows with descriptive text prompts to generate instrument-category similarity scores.
Feature-based MIR	Audio feature extraction and numerical comparison modules	Represents acoustic and musical characteristics and supports similarity-based retrieval and related analysis.
Data exchange	HTTP multipart upload and JSON responses	Transmits audio submissions and analysis results between the frontend and backend where the workflows are connected.

Layer	Technology	Purpose
Instrument-analysis API	`GET /`, `GET /api/health`, and `POST /api/analyze`	Provides the documented application entry point, health check, and instrument-analysis operation.
Development and execution environment	Python and JavaScript dependency environments	Provides the runtime environment for the backend, frontend, audio libraries, and pretrained models. Exact versions should be recorded from the final environment.

The table distinguishes the technologies associated with the primary instrument-analysis API from the broader feature-based MIR workflow. The presence of a library or module in the project does not, on its own, establish that every analytical capability is exposed through the same endpoint. Integration status should be documented according to the actual execution path.

For reproducibility, the final project submission should include the dependency specifications, model identifiers, required configuration, setup instructions, and the verified hardware environment. Any performance or accuracy claims should be supported by recorded experiments rather than inferred from the selected technologies.

CHAPTER 4

4. DESIGN APPROACH AND DETAILS

This chapter describes the system architecture, design methodology, component responsibilities, data representation, and dataset considerations of the proposed AI-Based Music Instrument Recognition System. The system combines audio preprocessing, source separation, pretrained audio-text similarity modelling, and feature-based music analysis to support instrument recognition and music similarity analysis.

The design consists of two related but distinct workflows. The primary instrument-recognition workflow processes an uploaded audio file through a FastAPI backend and returns instrument similarity scores and temporal analysis. The complementary Music Information Retrieval (MIR) workflow extracts musical and spectral features for music characterization and similarity-based retrieval. These workflows are documented separately to distinguish the primary instrument-analysis API from the broader feature-analysis and retrieval functionality.

4.1 System Architecture

The system follows a client-server architecture. The React frontend provides the interface through which users upload audio files and view analysis results. The FastAPI backend handles request validation, audio processing, model execution, and response generation.

The primary workflow begins when the user submits an audio file to the `/api/analyze` endpoint. The backend validates the file extension and size, decodes the audio, converts it to mono at a sampling rate of 48 kHz, and invokes the Demucs `htdemucs` source-separation model. The separated `other.wav` stem is then divided into one-second windows. A pretrained CLAP model, `laion/clap-htsat-unfused`, compares the audio windows against text prompts representing supported instrument categories.

The window-level similarity scores are aggregated to obtain song-level instrument scores. The backend also prepares temporal instrument information and window-coverage data before returning a JSON response to the frontend. The frontend presents the resulting analysis to the user.

The architecture accepts MP3, WAV, and FLAC files, with a maximum upload size of 100 MB. Separating the frontend from the backend processing allows the audio-analysis pipeline to be maintained independently of the user interface.

Figure 4.1 presents the overall system architecture and the relationship between the user interface, backend services, audio-processing components, pretrained models, and analysis response. It also distinguishes the primary instrument-recognition workflow from the complementary MIR workflow.

4.1.1 Model Coordination and Score Balancing

The instrument-recognition pipeline coordinates audio decoding, source separation, seg-

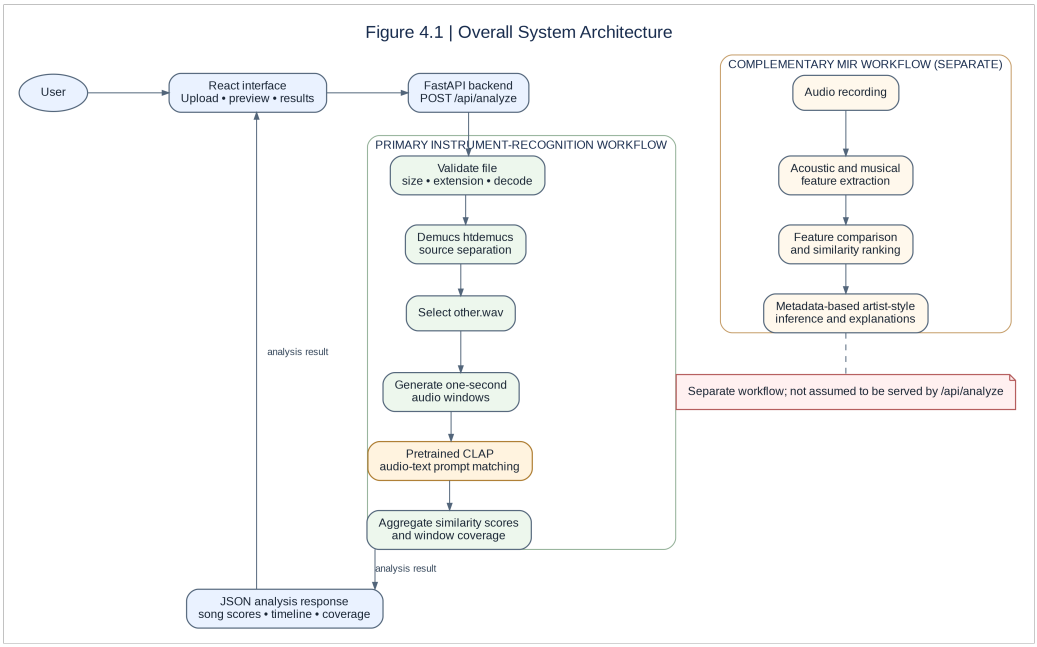


Figure 1: Overall System Architecture

mentation, model inference, and score aggregation. Each stage transforms the input into a representation suitable for the next stage.

Audio decoding standardizes the uploaded file for subsequent processing. The audio is represented as a mono waveform sampled at 48 kHz. Source separation is performed using Demucs with the `htdemucs` configuration. The `other.wav` stem is selected as the input to the subsequent instrument-analysis stage.

The selected audio is divided into one-second windows. When the final window is shorter than one second, padding is applied so that it can be processed consistently. Windows are evaluated in batches, with a default batch size of 16. This supports inference over longer recordings without requiring the entire recording to be evaluated as one input segment.

The pretrained CLAP model compares each audio window with text descriptions associated with the instrument categories. The resulting similarity values are recorded for individual windows and aggregated to produce song-level instrument scores. The system also derives temporal information indicating how instrument-related scores vary throughout the recording.

The model is used in evaluation mode, and the implemented pipeline does not contain a model fine-tuning loop or optimizer. Consequently, the reported scores represent model-based similarity indices rather than calibrated probabilities. They should not be interpreted as exact percentages of the physical contribution of each instrument to the original recording.

The main API returns the aggregated similarity indices and temporal analysis information. Other score-transformation components, including top-k or softmax-based percentage calculations and combined score representations, should be treated as separate components unless

their execution is established in the primary endpoint implementation.

4.1.2 MIR Feature Analysis and Retrieval Workflow

In addition to instrument recognition, the project includes a complementary Music Information Retrieval workflow designed to characterize musical recordings and support similarity-based retrieval.

The feature-analysis stage can extract characteristics such as estimated tempo, musical key and mode, harmonic and percussive energy, spectral centroid, spectral roll-off, spectral bandwidth, spectral flatness, Mel-frequency cepstral coefficients (MFCCs), root-mean-square energy, dynamic range, and zero-crossing rate. These features describe different aspects of a recording, including rhythmic, tonal, spectral, and energy-related properties.

The workflow can use these features to construct a unified representation of a song and compare that representation with entries in a reference catalogue. Similarity scores can be used to rank candidate recordings. Rule-based or heuristic logic may also derive genre and mood labels from selected features. Artist-style inference depends on the availability and quality of reference recordings and associated metadata.

These outputs require careful interpretation. Feature-derived labels are heuristic unless supported by a separately trained and evaluated classifier. Similarity rankings indicate relative resemblance to reference entries rather than calibrated probabilities. Artist-style inference depends on the reference catalogue and its metadata.

The MIR workflow is documented separately from the primary instrument-recognition API. Its inclusion describes the project's broader analytical scope without assuming that every feature-extraction, classification, and retrieval operation is executed by the `/api/analyze` endpoint.

Figure 4.7 illustrates the complementary MIR workflow, from audio input and feature extraction to feature representation, comparison with reference recordings, and interpretation of retrieval results.

4.2 Design

The system design describes the processing sequence, interactions between components, and organization of the analysis data. The diagrams in this section represent the workflow from different perspectives. The activity diagram emphasizes processing steps and decision points, the use-case diagram describes user-facing functionality, the backend class diagram summarizes component responsibilities, and the sequence diagram represents the order of interactions during an analysis request.

4.2.1 Data Flow Diagram

The primary data flow begins with an audio file submitted through the React interface. The frontend sends the file to the FastAPI backend through the `POST /api/analyze` endpoint.

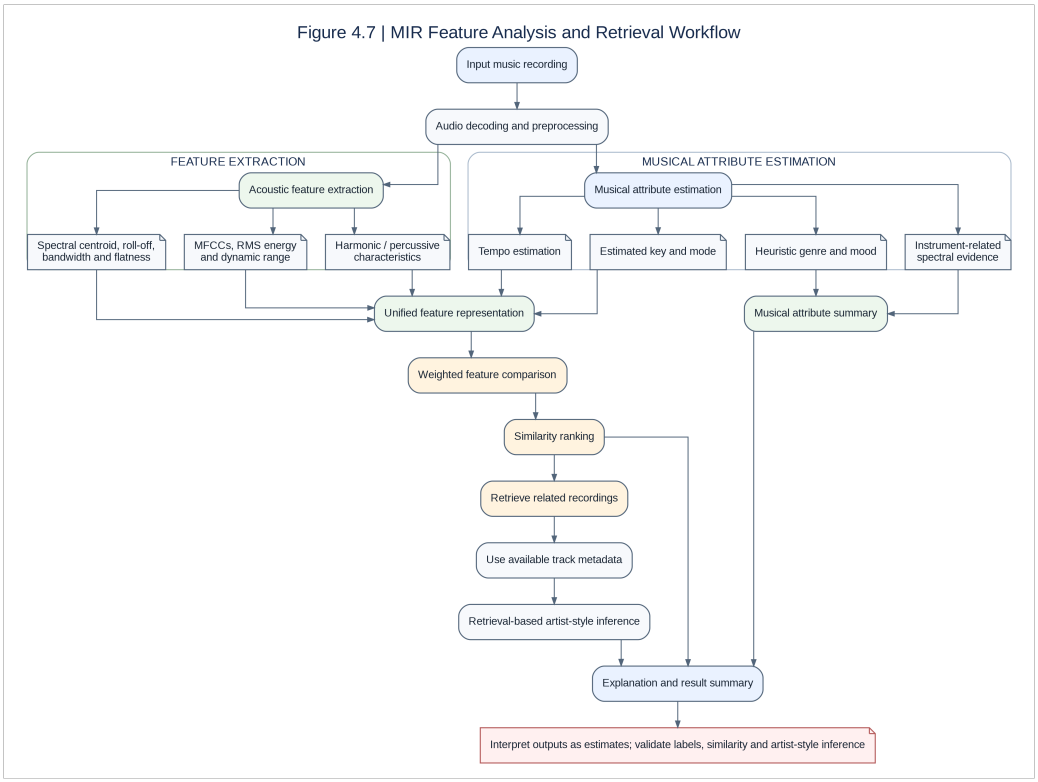


Figure 2: MIR Feature Analysis and Retrieval Workflow

The backend validates the upload, decodes the audio, performs source separation, divides the selected stem into windows, and runs pretrained CLAP similarity inference.

The model output is collected at the window level and aggregated into song-level scores. Temporal scores and window-coverage information are also prepared. These results are assembled into a structured JSON response and returned to the frontend for display.

The principal stages are input submission, validation, decoding, source separation, segmentation, model inference, score aggregation, response construction, and result presentation. Errors encountered during request handling or processing may prevent a valid analysis response from being produced; the exact error response depends on the implementation.

Figure 4.2 presents the activity sequence for instrument analysis, showing how uploaded audio progresses through the processing stages before the results are returned. It complements the higher-level architecture in Figure 4.1 by emphasizing the order of processing activities.

4.2.2 Use-Case Diagram

The system is designed around the user's interaction with the audio-analysis interface. The primary use case is to upload an audio recording and obtain an instrument-analysis response. The user can then inspect the returned song-level scores and temporal analysis information through the frontend.

The backend also exposes supporting endpoints. The root endpoint, `GET /`, provides a basic service response, while `GET /api/health` provides a health-check interface. The main

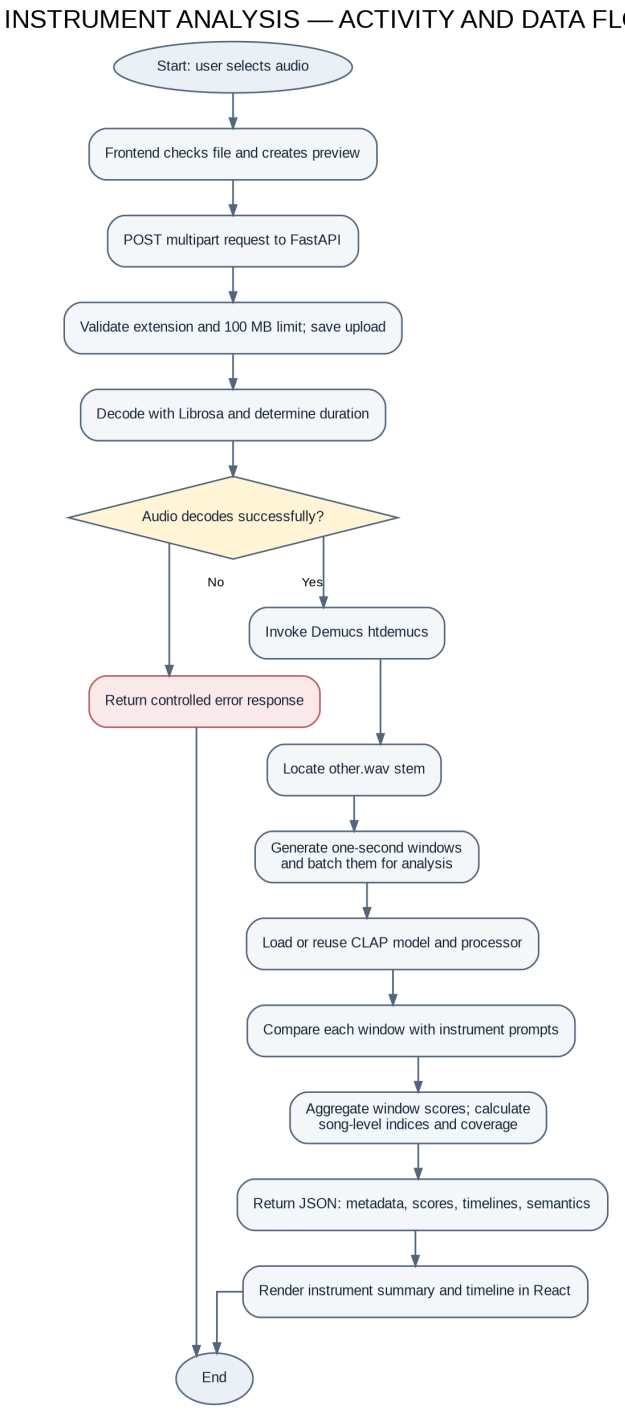


Figure 3: Instrument-Analysis Activity Diagram

analysis operation is handled by `POST /api/analyze`. These endpoints serve different purposes: service identification, health checking, and audio analysis.

The use-case view focuses on externally observable operations rather than internal model implementation. Audio decoding, source separation, windowing, model inference, and score aggregation are internal processing responsibilities rather than independent user actions.

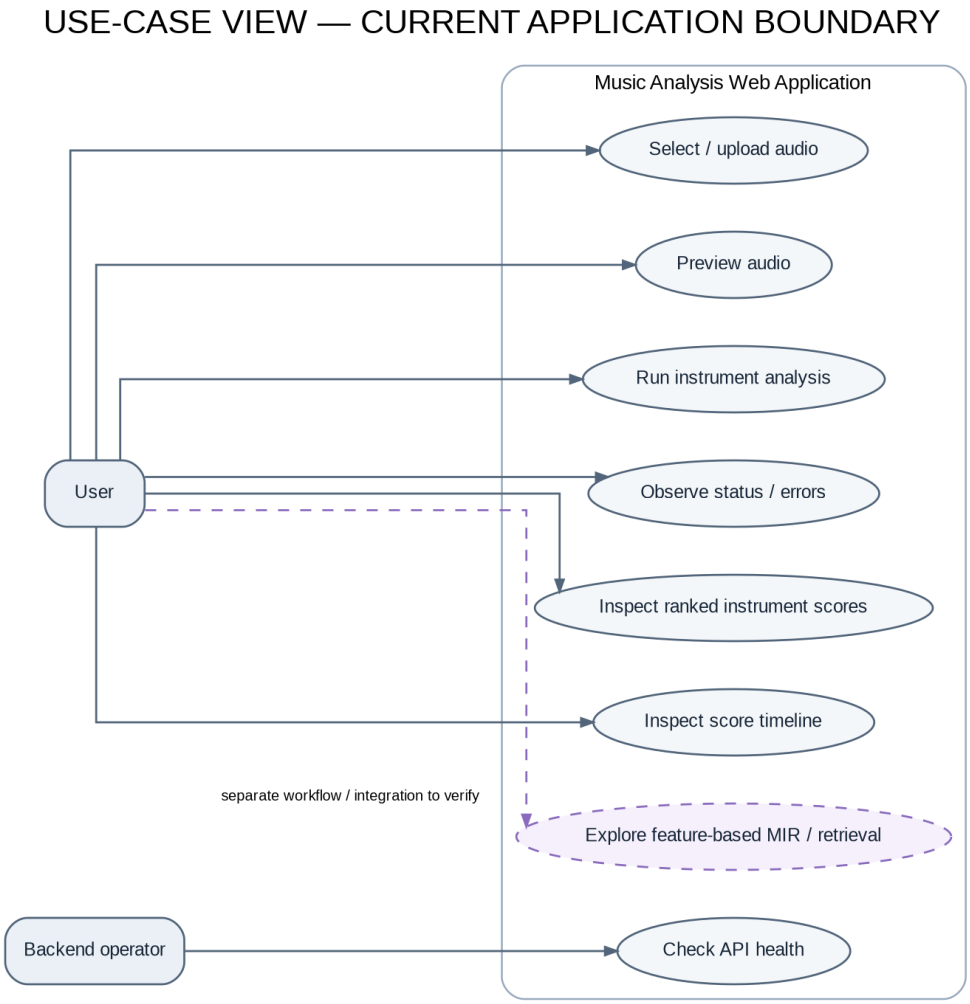


Figure 4: System Use-Case Diagram

Figure 4.5 presents the relationship between the user and the system's main externally accessible functions. It separates the user-facing analysis operation from the supporting service and health-check endpoints.

4.2.3 Backend Class Diagram

The instrument-analysis implementation is organized around components responsible for different stages of the pipeline. These responsibilities include accepting and validating requests, decoding and preparing audio, invoking source separation, segmenting the selected audio, running model inference, aggregating scores, and constructing the response.

Separating responsibilities improves modularity. Audio preprocessing can be examined independently of the user interface, and model inference can be considered separately from response formatting. This organization also makes it easier to identify which stage is responsible when an input fails to produce a usable analysis result.

The diagram represents the logical organization of the backend processing components. It should not be interpreted as a complete specification of every class attribute, method signature, or dependency in the source code.

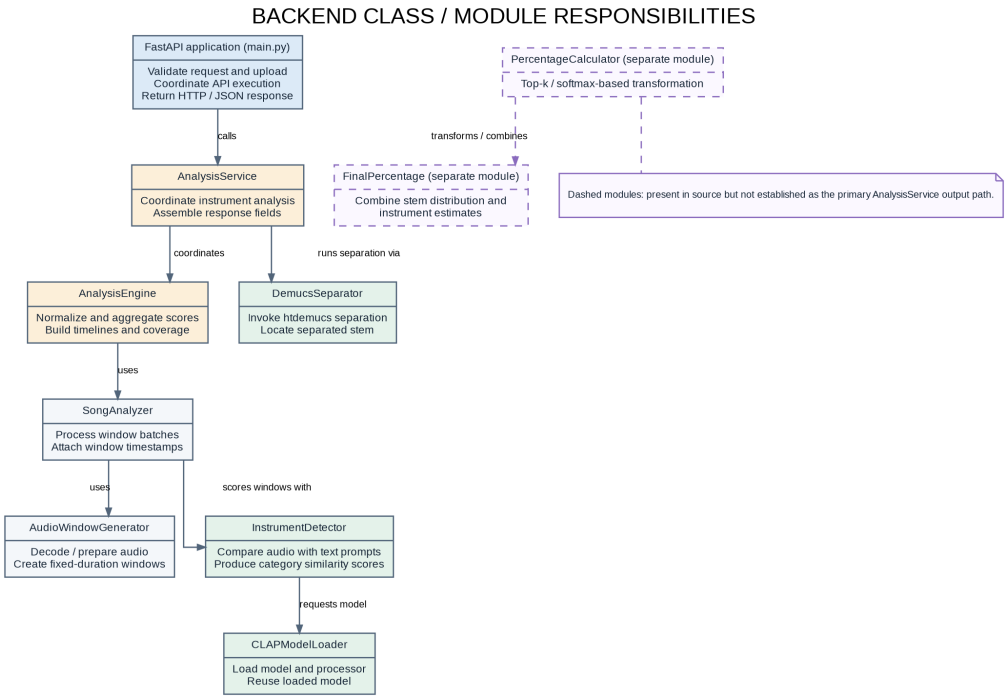


Figure 5: Backend Class Diagram

Figure 4.4 summarizes the logical organization of the instrument pipeline and the responsibilities associated with its principal backend components. It provides a component-oriented view of the processing design, complementing the workflow and interaction diagrams.

4.2.4 Sequence Diagram

The sequence diagram describes the interactions that occur when a user initiates an audio-analysis request. The React frontend submits the selected file to the FastAPI endpoint. The backend validates the request and passes the audio through the required preprocessing and model-inference stages.

After audio preparation, the source-separation stage produces the stems required by the analysis pipeline. The selected stem is segmented into windows, and the pretrained CLAP model evaluates the windows against instrument-related text prompts. The resulting scores are collected and aggregated by the analysis logic. The backend then assembles the response and returns it to the frontend.

This sequence emphasizes the order of operations and the exchange of data between the frontend and backend processing components. The user interface receives the completed analysis response rather than directly executing the audio-processing and model-inference operations.

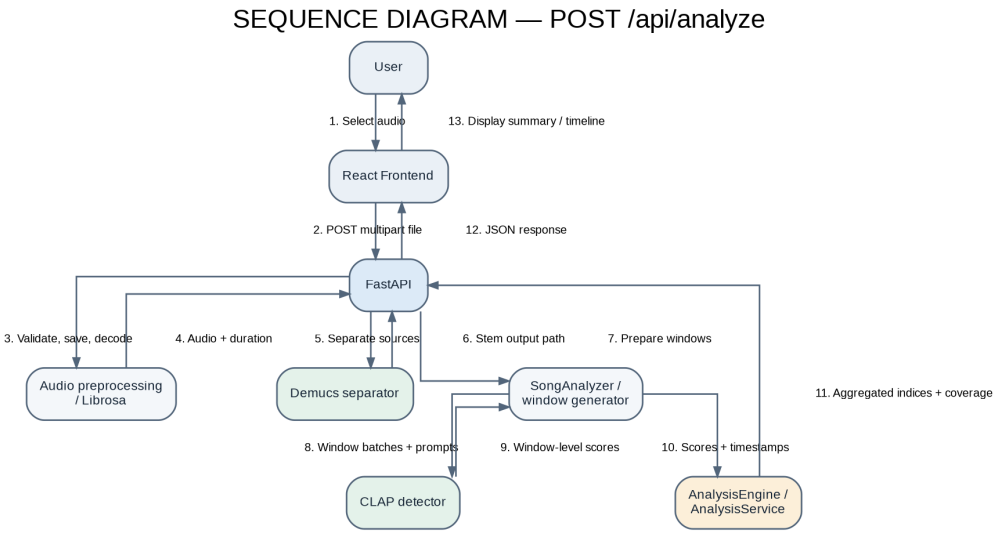


Figure 6: Sequence Diagram for Audio Analysis

Figure 4.3 illustrates the sequence of interactions involved in an analysis request, from submission by the frontend to processing by the backend and delivery of the final response. It complements Figure 4.2 by emphasizing component interactions rather than the control flow alone.

4.3 Data Representation

The primary analysis response is represented as structured JSON. It contains identifying information about the analyzed recording, window-level analysis results, aggregated song-level instrument scores, temporal instrument information, and coverage information.

The song metadata includes a generated identifier, the uploaded filename, and the recording duration. Window-level analysis records contain a window index, start and end times, and a dictionary of instrument similarity scores. The song-level score dictionary summarizes the results across the recording. Temporal information represents how instrument-related scores vary over time, while window-coverage information describes the extent of the recording represented by the analysis.

The response also contains compatibility aliases intended to support access to equivalent information through more than one field name. The exact structure and field names are determined by the implemented response schema.

The scores require careful interpretation. They are derived from pretrained audio-text similarity estimates and their aggregation. They are not automatically calibrated probabilities, and they do not directly measure the proportion of the original recording physically produced by

each instrument. A high score indicates stronger relative model similarity to a corresponding instrument description under the implemented scoring procedure.

The response structure separates per-window information from song-level summaries. This allows a consumer of the API to examine both the overall result and its variation over time.

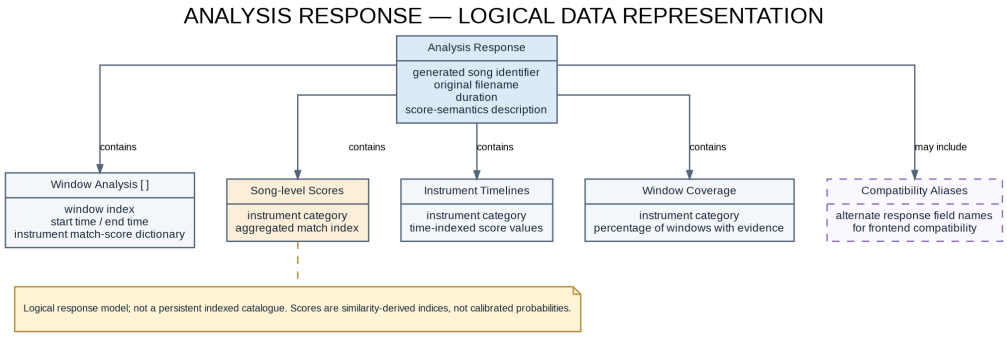


Figure 7: Analysis Response Data Structure

Figure 4.6 shows the main organization of the analysis response and the relationship between recording metadata, window-level scores, song-level scores, temporal information, and coverage data. The diagram provides a conceptual representation of the response rather than a replacement for the exact API schema.

4.4 Dataset Description

The system accepts audio recordings in MP3, WAV, and FLAC formats. Uploaded files are subject to a maximum size of 100 MB. After validation, the audio is decoded and converted to mono at a sampling rate of 48 kHz for the primary instrument-recognition workflow.

The processing pipeline uses pretrained models for source separation and audio-text similarity estimation. Demucs with the `htdemucs` configuration is used for source separation, and the pretrained `laion/clap-htsat-unfused` model is used to compare audio windows with instrument-related text prompts. These models are used in evaluation mode rather than being trained from scratch as part of each analysis request.

The project also considers a broader music collection for feature-based analysis and similarity retrieval. The estimated collection size has been described as approximately one million original songs with four versions per song, corresponding to a potential total of approximately four million audio files. This figure represents an estimate of the intended or considered collection size; it does not confirm that all files have been collected, processed, indexed, or evaluated.

A complete dataset inventory should identify the source of the recordings, file formats, version definitions, metadata availability, licensing conditions, and actual processing status. These details are important for reproducibility and for determining whether the reference collection is suitable for evaluation and similarity retrieval.

The available implementation details do not establish a verified dataset-wide instrument-recognition accuracy, a completed processing count for the estimated collection, or a benchmark against a labelled test set. Such results should be reported only after the corresponding experiments have been performed and their methodology and measurements documented.

CHAPTER 5

5. METHODOLOGY AND TESTING

5.1 Methodology

5.1.1 Audio Upload and Validation

The API accepts .mp3, .wav, and .flac extensions. It streams the request body to disk in chunks, rejects files exceeding 100 MB, rejects empty uploads, and attempts to decode the file to determine its duration. Unsupported extensions return HTTP 400, oversized files return HTTP 413, and decode failures return HTTP 422.

5.1.2 Source Separation

Demucs is invoked through a subprocess using the htdemucs model. The backend expects an other.wav file in the model output directory and uses that stem as the input to CLAP analysis. The output depends on Demucs and the quality of the source recording; source separation is an estimate and may leave interference between stems.

5.1.3 Windowing and Instrument Matching

Librosa loads the selected stem as mono audio at 48,000 samples per second. Audio is divided into one-second windows. The final window is padded with zeros if it is shorter than one second. SongAnalyzer processes windows in batches of 16 by default. InstrumentDetector contains descriptive prompt sets for categories including guitar, drums/percussion, piano/keyboard, strings, brass, woodwinds, and bass. CLAP similarity is adjusted against a generic-music baseline and a minimum evidence threshold; it is not a supervised probability estimate.

5.1.4 Aggregation

AnalysisEngine clips scores to the range 0–1 and calculates a mean score per instrument across all windows, multiplying the mean by 100. It also computes window coverage, defined as the percentage of windows with a non-zero score for that instrument. This coverage measures how widely the model returns evidence across windows; it is not a calibrated probability that the instrument is present.

5.1.5 Frontend Processing

The React application provides file selection, a browser-side waveform preview generated from decoded audio samples, audio playback controls, analysis status messages, and a result view. The frontend calls the backend endpoint through a configurable base URL and normalises common response shapes. The interface includes demonstration data for the initial preview; demonstration values must not be presented as output from an uploaded song.

5.2 Testing Strategy

The testing strategy is designed to verify the functional correctness, reliability, and consistency of the AI-Based Music Instrument Recognition System. Since the system combines file handling, audio preprocessing, source separation, pretrained model inference, score aggregation, and frontend visualisation, verification must consider both individual components and the complete upload-to-results workflow.

The testing approach is divided into functional verification, audio-pipeline verification, frontend–backend integration, and model evaluation. These activities serve different purposes: functional verification checks whether the application behaves as intended, pipeline verification checks intermediate audio-processing operations, integration verification checks communication between components, and model evaluation measures recognition quality against labelled reference data.

The following subsections define the verification scope and evaluation methodology. They describe the checks required to establish system reliability and do not imply that every test has already been executed successfully.

5.2.1 Functional Verification

Functional verification focuses on the application's externally observable behaviour and the main use cases supported by the implementation.

Audio Upload and Validation: The backend accepts MP3, WAV, and FLAC files and enforces a maximum upload size of 100 MB. Verification should confirm that supported files are accepted and that unsupported extensions, empty uploads, oversized files, and undecodable audio are handled appropriately. The expected error responses include HTTP 400 for unsupported file types, HTTP 413 for files exceeding the upload limit, and HTTP 422 for audio that cannot be decoded, as specified by the implementation.

API Behaviour: The root endpoint, `GET /`, should be checked for its service response, while `GET /api/health` should be checked for the expected health-status response. The `POST /api/analyze` endpoint should be verified using supported audio files and invalid inputs. Verification should examine the HTTP response, response structure, and whether processing failures are communicated without returning misleading analysis results.

Audio Analysis: The analysis workflow should be checked to ensure that a valid upload proceeds through audio decoding, Demucs source separation, selection of the `other.wav` stem, window generation, CLAP similarity estimation, score aggregation, and response construction. Where a processing stage fails, the resulting behaviour should be examined to ensure that incomplete processing is not presented as a successful analysis.

Frontend Interaction: The React interface should be checked for file selection, local audio playback, waveform preview, processing-status updates, error messages, and the presentation of the returned instrument scores and temporal information. The displayed results should correspond to the response received from the backend. Demonstration or preview data should remain distinguishable from results generated by analysing an uploaded recording.

5.2.2 Audio-Pipeline Verification

The audio pipeline contains several transformations that can affect the final instrument scores. Verification should therefore examine the intermediate outputs as well as the final response.

- **Audio decoding:** Confirm that supported files can be decoded and that their duration can be determined. Invalid or corrupted files should be handled through the defined error path.
- **Source separation:** Verify that the Demucs subprocess is invoked with the configured `htdemucs` model and that the expected `other.wav` output is available before the next processing stage begins.
- **Window generation:** Check that the selected audio is represented as mono audio sampled at 48 kHz and divided into one-second windows. The final partial window should be padded where necessary, and window timestamps should remain consistent with the recording duration.
- **Model inference:** Verify that the pretrained CLAP model and its processor are available and that window-level similarity scores are produced for the configured instrument prompts. Model loading failures and inference errors should be recorded for diagnosis.
- **Score aggregation:** Check that the aggregation logic bounds the window-level scores to the configured range and calculates song-level mean-score indices consistently. Window-coverage values should correspond to the implementation's definition based on windows with non-zero evidence after thresholding.
- **Response construction:** Verify that the response contains the expected recording metadata, window-level results, song-level scores, temporal information, and coverage fields. Timestamps and score arrays should remain internally consistent.

These checks are intended to identify errors introduced at individual processing stages. In particular, the presence of a valid JSON response alone does not establish that source separation, model inference, or score aggregation produced meaningful results.

5.2.3 Integration and Use-Case Verification

Integration verification examines the complete interaction between the React frontend and FastAPI backend. The primary use case begins when a user selects an audio file, previews it, initiates analysis, observes the processing status, and inspects the returned instrument scores and time-based visualisation.

The verification process should confirm that the frontend submits the file in the expected request format, the backend processes the request, and the returned JSON is normalised and displayed correctly. It should also check that invalid inputs and backend failures produce appropriate feedback rather than leaving the interface in an indefinite processing state.

The following user-facing behaviours should be covered:

- Selecting a supported audio file and previewing its playback.
- Initiating analysis and displaying processing-status information.
- Receiving and presenting ranked instrument match indices.
- Displaying the time-based variation of instrument scores.
- Handling invalid files, unsuccessful requests, and incomplete responses.
- Distinguishing actual analysis output from demonstration or preview data.

The supporting service and health-check endpoints should be verified independently. Administrative dataset ingestion and artist-style search are not included as primary API use cases because they are not exposed by the supplied backend implementation.

5.2.4 Model Evaluation Methodology

Functional correctness and instrument-recognition quality must be evaluated separately. A system may execute its pipeline correctly while still producing inaccurate or ambiguous instrument estimates. Consequently, successful request handling cannot be used as evidence of recognition accuracy.

A quantitative evaluation should use a labelled collection of audio recordings for which the instrument categories present in each recording are known. The predicted instrument categories should be compared with these reference labels using precision, recall, and F1-score. Precision measures the proportion of positive predictions that are correct, recall measures the proportion of reference-positive cases identified by the system, and F1-score provides their harmonic mean.

For multi-instrument recordings, evaluation should account for the presence of multiple valid labels per recording. Results should be reported per instrument category as well as across categories using a clearly specified aggregation method. The decision threshold used to convert similarity scores into positive instrument predictions should be documented and selected using validation data rather than the final test set.

The dataset should be divided into training, validation, and test partitions where model development or threshold selection requires them. The split should be performed by original song identity so that alternate versions or covers of the same composition do not appear across multiple partitions. This reduces the risk of data leakage and overly optimistic evaluation.

Additional evaluation should examine processing latency, failure rates, and robustness to differences in audio duration, recording quality, and musical arrangement. Latency measurements should identify the hardware and software environment, audio duration, and whether model-loading time is included. Robustness results should be reported using defined test conditions rather than subjective observations.

The instrument scores produced by the current pipeline are similarity-derived indices and are

not calibrated probabilities or direct measurements of each instrument's physical contribution to the original mix. They should therefore be evaluated against labelled reference data before being interpreted as estimates of actual instrument presence or contribution.

5.2.5 Evidence and Reporting of Results

Testing outcomes should be documented using reproducible evidence, including test inputs, expected behaviour, observed behaviour, execution status, and relevant logs or response samples. Failures should be recorded with enough detail to identify the affected component and support subsequent debugging.

At the time of preparing this report, the supplied project materials do not establish complete runtime test results, a labelled evaluation set, or measured instrument-recognition accuracy. No quantitative success rate or benchmark is therefore claimed in this section. The verification procedures described above define the checks required to assess the implemented system and provide a basis for reporting measured results after execution.

The final evaluation should distinguish three forms of evidence: functional correctness of the application, consistency of the audio-processing pipeline, and recognition quality measured against labelled data. Keeping these dimensions separate provides a more reliable assessment of the system than treating a successful API response as proof of accurate instrument recognition.

5.3 Evaluation Metrics for Further Validation

A labelled test set should be used to calculate per-class precision, recall, F1-score, and confusion matrices. Multi-label instrument detection should use a threshold chosen on validation data. Robustness tests should include short clips, quiet instruments, dense arrangements, different genres, noisy recordings, and audio with multiple versions of the same composition. Latency should be measured separately for decoding, Demucs separation, model loading, inference, aggregation, and end-to-end response time.

5.4 Fine-Tuning and Weight Optimisation

The inspected code demonstrates pretrained-model loading and deterministic score aggregation, not parameter fine-tuning. A reproducible fine-tuning workflow would require a labelled dataset, a documented train/validation/test split, a loss function appropriate to multi-label audio tagging or contrastive learning, an optimizer and learning-rate schedule, checkpoint selection, and a comparison with the pretrained baseline. Feature-weight balancing would require a defined feature vector and a validation objective. These elements should be added and measured before the report claims that model weights were fine-tuned or that the system achieved a particular accuracy.

5.5 Test Limitations

Source inspection is not equivalent to executing the application. The ZIP files contain implementation code but do not provide reproducible logs showing a complete successful

run, benchmark measurements, or labelled accuracy results. Those results must be collected on the target machine before the report is submitted as experimentally validated.

CHAPTER 6

6. PROJECT DEMONSTRATION

This chapter describes the demonstration workflow of the AI-Based Music Instrument Recognition System, focusing on the interaction between the React frontend and the FastAPI backend. The demonstration is intended to show how a user submits an audio recording, follows the analysis process, and examines the instrument-related results returned by the system.

The application follows an upload-to-results workflow. The user selects a supported audio file and can preview the recording in the browser before initiating analysis. The frontend submits the file to the backend, where it undergoes validation, audio decoding, source separation, window generation, and pretrained CLAP-based instrument matching. The backend aggregates the resulting similarity scores and returns a structured JSON response. The frontend then presents the available instrument match indices and their temporal variation.

The demonstration covers the implemented instrument-analysis workflow. It does not establish the performance of a large-scale indexed music catalogue or the completion of additional music-analysis features that are not exposed through the supplied backend API.

6.1 Demonstration Environment

The application consists of two principal components: a React frontend and a FastAPI backend. The frontend provides the user interface for file selection, playback, waveform preview, processing-status updates, and result visualisation. The backend handles upload validation, audio preprocessing, source separation, model inference, score aggregation, and response generation.

The backend requires the configured Python environment and the dependencies used by the audio-processing pipeline, including Librosa, Demucs, and the pretrained CLAP model implementation. The frontend requires its configured JavaScript environment and development server. Both components must be running and configured to communicate through the correct API base URL.

Before beginning the demonstration, the backend health endpoint should be checked to confirm that the service is responding. The frontend should then be opened in the browser and checked for successful communication with the backend.

6.2 Demonstration Workflow

The demonstration proceeds through the following stages.

6.2.1 Application Initialisation

The FastAPI backend is started in the configured Python environment. The `/api/health` endpoint is accessed to check whether the service responds as expected. The React development server is then started using the project's configured development command.

The frontend API base URL must point to the running backend. Incorrect configuration can prevent requests from reaching the analysis endpoint even when the interface itself loads correctly.

6.2.2 Audio Selection and Preview

The user selects an audio recording in MP3, WAV, or FLAC format, subject to the maximum upload size of 100 MB. The frontend provides a local playback and waveform-preview facility, allowing the user to inspect the selected recording before submitting it for analysis.

The preview is generated in the browser and should be distinguished from the results produced by the backend. Any sample values displayed in the initial demonstration interface must not be interpreted as analysis results for the selected recording.

6.2.3 Backend Audio Processing

When the user initiates analysis, the frontend submits the audio file to the `POST /api/analyze` endpoint. The backend validates the request and attempts to decode the uploaded audio. It then invokes Demucs using the `htdemucs` configuration and selects the generated `other.wav` stem for instrument analysis.

The selected audio is converted to mono at 48 kHz and divided into one-second windows. The final partial window is padded when necessary. The pretrained CLAP model compares the audio windows with text prompts associated with supported instrument categories. The resulting window-level similarity scores are collected and aggregated into song-level match indices, temporal information, and window-coverage values.

The duration of the analysis may vary depending on the input recording, source-separation processing, model availability, and execution environment. The demonstration should therefore observe the actual processing behaviour rather than assume a fixed response time.

6.2.4 Results Visualisation

After successful processing, the backend returns a JSON response containing recording metadata, window-level analysis, song-level instrument scores, temporal information, and coverage data. The frontend normalises the response and presents the available analysis results.

The song-level summary provides an overall view of the model's instrument-related similarity scores. The temporal view allows the user to inspect how these scores vary across the recording. Together, these views provide both a recording-level summary and a time-localised representation of the model output.

The displayed scores are similarity-derived indices. They must not be interpreted as calibrated probabilities or as exact percentages of each instrument's physical contribution to the original audio mix.

6.3 Demonstration Checklist

The following checklist defines the steps required for a reproducible demonstration.

1. Activate the configured Python environment and start the FastAPI backend.
2. Access `GET /api/health` and verify the observed response.
3. Start the React frontend using the project's configured development command.
4. Confirm that the frontend API base URL points to the running backend.
5. Select a supported MP3, WAV, or FLAC recording under the 100 MB limit.
6. Check that local playback and waveform preview behave as expected.
7. Submit the recording for analysis and observe the processing-status messages.
8. Inspect the backend logs for upload validation, source separation, model inference, and response generation.
9. After a successful response, review the song-level instrument scores and temporal analysis view.
10. Submit an unsupported file to verify the expected input-validation behaviour.
11. Record any errors, unexpected responses, or incomplete processing for subsequent investigation.
12. Capture screenshots of the live application and its actual output for inclusion in the report.

The checklist should be completed using the running application. Screenshots should show genuine application behaviour, and any reported observations should correspond to the actual recording and response used during the demonstration.

6.4 Current Demonstration Scope and Limitations

The supplied implementation supports the primary workflow from audio upload through instrument analysis and frontend result presentation. Its demonstrated capabilities should be described in terms of the functionality present in the React and FastAPI source code.

The initial frontend view contains demonstration values for preview purposes. These values are not evidence of model performance and should not be presented as results from an uploaded recording. Actual analysis results depend on a successful backend response and the completion of the processing pipeline.

The current implementation also has limitations that affect how the demonstration should be interpreted. Demucs produces estimated source stems rather than perfect instrument isolation, and CLAP generates similarity-based scores rather than calibrated probabilities. The selected `other.wav` stem may not retain all instrument evidence present in the original mixture. These factors can influence the final scores and their temporal distribution.

The available source does not establish a complete deployed catalogue-search workflow or a fully evaluated implementation of additional genre, mood, tempo, key, and artist-style outputs. These capabilities should not be presented as completed features of the demonstrated application unless their implementation and execution can be independently verified.

Furthermore, a successful demonstration establishes that the tested workflow operates under the observed conditions; it does not by itself establish instrument-recognition accuracy, robustness across musical genres, or performance at the estimated dataset scale. Those claims require separate experiments and documented evaluation results.

6.5 Demonstration Evidence

The final demonstration record should include screenshots of the application interface, the selected audio file, the processing state, and the actual results returned by the backend. Where appropriate, supporting evidence may include the health-endpoint response, relevant backend logs, and an example analysis response with identifying or sensitive information removed.

Each screenshot should have a concise caption explaining what it demonstrates. For example, an upload screenshot can document the input stage, a processing screenshot can document the analysis workflow, and a results screenshot can show the returned instrument scores and temporal view.

Only evidence collected from the running application should be used to substantiate claims about observed behaviour. If a stage could not be executed, the report should identify it as unverified rather than implying successful completion.

CHAPTER 7

7. RESULT AND DISCUSSION

The AI-Based Music Instrument Recognition System establishes an integrated workflow for analysing uploaded music recordings and presenting instrument-related evidence through a web interface. The implementation combines file validation, audio preprocessing, source separation, pretrained audio-text similarity matching, temporal segmentation, score aggregation, and frontend visualisation. These components address different stages of the analysis problem, from accepting a valid recording to presenting the model output in a form that can be inspected by the user.

The significance of the implementation lies in the integration of these stages into a modular application rather than in a claim of experimentally established recognition accuracy. The available source code establishes the presence of the principal processing components and their intended interaction. However, source inspection alone does not demonstrate that every execution path operates successfully under all conditions, nor does it establish the accuracy with which the system identifies instruments in previously unseen recordings.

This chapter discusses the implementation findings, the interpretation of the generated scores, the implications of the proposed dataset scale, the requirements for a defensible cost analysis, and the limitations that affect the current system. It also identifies the technical developments needed to move from a functioning analysis workflow towards a quantitatively evaluated music-analysis platform.

7.1 Results Supported by Source Inspection

The principal result established by the supplied implementation is the development of a modular audio-analysis pipeline connecting a web-based interface with a backend processing service. The backend is implemented using FastAPI, while the frontend uses React. Their separation assigns user interaction and result presentation to the frontend and audio processing, model execution, and response generation to the backend.

The input-handling stage accepts MP3, WAV, and FLAC files and applies a maximum upload-size limit of 100 MB. Empty uploads and unsupported or undecodable inputs are handled through validation and error paths. This is an important part of the system because the reliability of subsequent processing depends on receiving an audio file that can be decoded and analysed. Input validation also prevents invalid requests from being treated as successful analysis operations.

After validation, the backend invokes Demucs using the `htdemucs` configuration. The implementation expects the generated `other.wav` stem and uses it as the input to the subsequent instrument-matching stage. This establishes source separation as an explicit part of the analysis pipeline. It also introduces a dependency: the quality and availability of the selected stem affect the information available to the instrument detector.

The selected audio is loaded as mono audio at a sampling rate of 48 kHz and divided into

one-second windows. The final partial window is padded when necessary, and windows are processed in batches, with a default batch size of 16. This segmentation strategy provides a consistent unit of analysis and preserves temporal information that would be lost if the system returned only one score for the entire recording.

Instrument matching is performed using the pretrained `laion/clap-htsat-unfused` model. The implementation compares audio windows with descriptive prompts for instrument categories, including guitar, drums and percussion, piano and keyboard, strings, brass, woodwinds, and bass. The resulting similarity evidence is processed by the aggregation logic to produce song-level instrument match indices and window-level information.

The backend response includes recording metadata, window-level analysis, song-level scores, instrument timelines, and window-coverage information. This structure supports two complementary forms of inspection: an overall summary of the model's instrument-related evidence and a time-based view showing how that evidence varies throughout the recording.

The frontend provides file selection, local audio playback, waveform preview, processing-status information, error handling, and result visualisation. These capabilities connect the analysis pipeline to a user-facing workflow rather than leaving the processing logic as an isolated backend operation.

Taken together, the source establishes an implementation that covers the major stages of an upload-to-analysis application. The evidence supports the existence of the processing logic and the designed data flow. It does not, by itself, establish that every stage has passed runtime testing, that the system identifies instruments accurately, or that the complete pipeline has been benchmarked. Those conclusions require execution evidence and a labelled evaluation dataset.

7.2 Interpretation of Instrument Scores and Temporal Evidence

The interpretation of the generated scores is central to understanding the system's output. The current implementation uses pretrained audio-text similarity matching rather than a separately fine-tuned, calibrated instrument-probability model. Its numerical output must therefore be understood in terms of the implemented scoring procedure.

At the window level, the instrument detector compares an audio segment with descriptive text prompts associated with the supported instrument categories. The resulting similarity evidence is adjusted using a generic-music baseline and a minimum evidence threshold. The aggregation stage clips scores to the interval from zero to one, calculates a mean for each instrument across the analysed windows, and multiplies the mean by 100 to produce a song-level match index.

If an instrument receives a song-level score of 70, the value represents an aggregated model-derived match index on the implementation's reporting scale. It does not mean that the instrument has a 70% probability of being present. It also does not mean that the instrument

contributes 70% of the recording's loudness, energy, or physical sound content.

This distinction matters because audio-text similarity is influenced by the learned representation of the model, the wording of the text prompts, the characteristics of the recording, and the interference between overlapping sources. A strong match score may provide useful evidence for ranking candidate instrument categories, but the score cannot automatically be interpreted as a calibrated measure of instrument presence.

The temporal analysis provides additional information by retaining scores for individual audio windows. Rather than representing the entire recording through a single value, the system can show where instrument-related evidence is stronger or weaker over time. This is useful for inspecting recordings in which the arrangement changes between sections, although temporal variation in the scores should not automatically be interpreted as a definitive account of when an instrument physically enters or leaves the mix.

The system also computes window coverage, defined by the implementation as the percentage of analysed windows with non-zero evidence for an instrument after thresholding. Coverage and mean score describe different aspects of the output. The mean score summarizes the magnitude of the returned evidence, while coverage describes how widely that evidence occurs across the windows.

For example, two instrument categories could have similar mean scores but different coverage values if one receives stronger evidence in a small number of windows and the other receives weaker evidence over a larger portion of the recording. This distinction can help users inspect the temporal distribution of model evidence. It should not, however, be treated as a validated measure of musical prominence or instrument duration without comparison against reference annotations.

A quantitative evaluation is required to establish how closely these outputs correspond to actual instrument presence. Such an evaluation should compare predictions with labelled recordings, document the decision thresholds, report per-class results, and investigate common sources of error. Calibration should also be assessed separately if the intended application requires outputs that can be interpreted probabilistically.

7.3 Dataset Scale and Data Management

The project author reports an intended or considered collection of approximately one million original songs with four versions per song, corresponding to an estimated total of approximately four million audio files. The versions are described as including original recordings and instrument-specific versions, such as guitar or drum covers. This estimate indicates the potential scale of the broader data-management problem, but it does not establish that all files have been collected, validated, processed, or indexed.

The distinction between an intended dataset size and an operationally processed dataset is important. A large file collection introduces requirements beyond the inference procedure used for a single uploaded recording. Each file must be identifiable, its source and format

must be recorded, and its processing state must be tracked. Without this information, it becomes difficult to determine which recordings have been analysed, which failed, and which require reprocessing.

A suitable dataset manifest should include a stable track identifier, original filename, version type, source dataset, duration, sample rate, file location, licence information, available instrument labels, and processing status. Where multiple versions represent the same composition, the manifest should retain their relationship to the same original-song identifier. This enables duplicate detection and supports evaluation splits that prevent alternate versions of the same composition from appearing in both training and test partitions.

At this scale, processing should be designed as a controlled offline workflow rather than as an assumption that the interactive web endpoint will ingest the entire collection. A batch-processing system would benefit from queued jobs, persistent checkpoints, retry handling, structured failure logs, and monitoring of temporary files. These mechanisms would allow interrupted jobs to resume without unnecessarily repeating completed work and would make processing failures easier to investigate.

Storage requirements also depend on the source files, the generated stems, and any intermediate representations retained during processing. Temporary separation outputs should be managed according to an explicit retention policy. If extracted features or embeddings are retained for future retrieval, their schema, model version, preprocessing configuration, and source-file identity should be recorded so that the resulting data remains traceable.

The current web backend processes an uploaded file through an individual analysis request. It does not establish a persistent, indexed catalogue of approximately four million recordings. The larger dataset should therefore be treated as a scale objective requiring a separate ingestion, processing, and storage strategy. Before reporting the collection size as an achieved result, the actual file manifest, duplicate-removal procedure, source dataset names, and processing status must be verified.

7.4 Cost Analysis

A reliable cost analysis requires measured information about the workload, execution environment, storage requirements, and failure rate. The supplied project materials do not establish a verified bill of materials, cloud invoice, complete throughput benchmark, or average per-song processing time. Consequently, a numerical cost per recording or a total cost for processing the estimated collection cannot be justified from the available evidence.

The principal computational stages have different resource requirements. Audio decoding and window generation require CPU processing and memory. Demucs source separation can be computationally intensive, while CLAP inference depends on the model configuration and available CPU or GPU resources. Model-loading time, batch size, recording duration, and the hardware used can all affect the observed processing time.

A useful measurement procedure should record the time required for audio decoding, source

separation, model loading, window-level inference, score aggregation, and the complete request. These measurements should be collected separately so that the principal sources of latency can be identified. The evaluation should also record the audio duration, file size, hardware configuration, software environment, and whether the model was already loaded when the measurement began.

Storage cost should be estimated from the measured size of original recordings, retained stems, and any persistent feature or embedding representations. The calculation should distinguish temporary files from data retained for future use. It should also account for failed and retried jobs, since repeated processing can increase compute consumption without increasing the number of successfully processed recordings.

For a batch-processing workload, the compute cost per successfully processed recording can be estimated as:

$$C_{\text{song}} = \frac{C_{\text{compute}} + C_{\text{storage}} + C_{\text{data transfer}} + C_{\text{overhead}}}{N_{\text{successful}}}$$

where $N_{\text{successful}}$ represents the number of recordings that complete the defined processing workflow. The cost categories should be included only when they apply to the selected deployment environment and can be measured or reasonably estimated from documented rates.

A projection for the full collection should be based on measured average processing time and resource consumption, not on an assumed constant response time. The projection should account for the distribution of recording durations, the number of processing failures, the hardware configuration, and any parallelism used during batch processing.

The immediate requirement is therefore to establish a reproducible benchmark on a representative sample of recordings. Once average processing time, resource consumption, storage requirements, and failure rates have been measured, a defensible cost estimate can be prepared. Until then, any numerical cost projection would be speculative rather than an observed result of the project.

7.5 Limitations and Threats to Validity

The current implementation establishes a practical pipeline for producing instrument-related similarity scores, but several limitations affect the interpretation, reproducibility, and generalisation of its output.

Dependence on source separation: The instrument-analysis pipeline uses the `other.wav` stem produced by Demucs. Source separation is an estimated transformation of the original mixture and may leave interference between sources or alter information needed for subsequent identification. Selecting a single stem may also exclude useful evidence present in other stems. The impact of this design choice should be assessed by comparing predictions from the selected stem with predictions from the original mixture and, where appropriate,

alternative stem combinations.

Similarity scores are not calibrated probabilities: The CLAP model provides audio-text similarity evidence rather than a guaranteed probability that a particular instrument is present. Prompt wording, overlapping sounds, recording quality, and the composition of the musical arrangement can affect the returned values. Without a labelled evaluation set and an explicit calibration procedure, the reported indices cannot establish the probability of instrument presence or the physical contribution of each instrument to the original mix.

Temporal and coverage interpretation: Window-level scores and coverage values preserve information about the distribution of model evidence throughout the recording. However, they are not equivalent to verified instrument activity annotations. A window may contain evidence that matches an instrument prompt without providing sufficient information to determine the exact source, and weak evidence may result from masking or changes in the arrangement. Temporal claims should therefore be validated against labelled or manually annotated recordings.

Absence of demonstrated parameter fine-tuning: The supplied implementation demonstrates loading and using a pretrained CLAP model in evaluation mode. It does not establish that the model parameters are fine-tuned as part of the analysis workflow. The quality of the outputs must therefore be evaluated as the performance of the implemented pretrained-model pipeline, rather than presented as the result of a newly trained instrument-recognition model.

Incomplete experimental evidence: The available project materials do not establish a complete set of reproducible runtime logs, a labelled test set, or measured accuracy and latency results. Code inspection identifies the implemented processing stages, but it cannot establish that every supported input has been processed successfully or that the resulting instrument scores are accurate. Functional testing and quantitative model evaluation remain necessary before claims about reliability or recognition performance can be made.

Dataset verification and reproducibility: The estimated collection of approximately four million audio files requires verification against an actual manifest. The exact source datasets, licensing conditions, duplicate-removal procedure, label availability, and completed processing count have not been established by the supplied source files. These omissions limit the reproducibility of a large-scale evaluation and prevent the dataset estimate from being treated as a verified processing result.

Scope of the demonstrated system: The primary backend supports the upload-to-instrument-analysis workflow. The supplied implementation does not establish a complete deployed catalogue-search system or a fully evaluated implementation of genre, mood, tempo, key, artist-style inference, and explanation generation. These capabilities must be distinguished from the existing instrument-analysis functionality when describing the project's results.

These limitations identify the evidence required to strengthen the system rather than invalidate the implemented workflow. They also define the boundary between established

implementation details and conclusions that require further experimentation.

7.6 Future Enhancements

Future development should prioritise improvements that can be implemented, measured, and evaluated independently. This will allow the system to progress from a functional instrument-analysis application towards a broader and more rigorously evaluated music-information platform.

Improved instrument evidence and score calibration: The current pipeline could be evaluated against the original mixture and alternative source-separation outputs to determine whether the selected `other.wav` stem improves instrument matching. The score aggregation procedure could also be examined using labelled recordings to determine appropriate thresholds and calibration methods. Any changes should be compared against the current pretrained baseline using the same evaluation data.

Systematic evaluation and error analysis: A labelled multi-instrument dataset should be established with documented provenance, annotation rules, and stable song identifiers. The evaluation should report per-instrument precision, recall, F1-score, and appropriate multi-label summary metrics. Error analysis should identify which instrument categories are confused most often and whether performance changes with recording duration, arrangement density, audio quality, or instrument prominence.

Expanded music-feature analysis: Dedicated modules could be developed and evaluated for tempo, key and mode, rhythmic properties, harmonic content, timbral features, genre, and mood. Each module should have a defined input representation, output format, and validation method. Features derived through heuristic rules should be clearly distinguished from outputs produced by trained and evaluated classifiers.

Persistent feature storage and similarity retrieval: A separate catalogue-processing workflow could store validated song metadata, extracted feature vectors, and audio embeddings. An approximate-nearest-neighbour index could then support efficient retrieval of recordings with similar representations. Such a system would require explicit catalogue ingestion, duplicate management, index maintenance, and retrieval evaluation. It should not be considered complete until the reference catalogue and search workflow have been implemented and tested.

Interpretable result presentation: The interface could be extended to explain why a candidate instrument or reference recording receives a strong match. Such explanations should link the result to observable model scores, temporal evidence, feature similarities, or reference examples. They should not claim causal certainty about an instrument's presence when the underlying evidence remains similarity-based.

Scalable offline processing: The estimated large-scale collection would benefit from a queue-based batch pipeline with stable identifiers, resumable checkpoints, retry policies, structured logging, and storage management. This processing infrastructure should be

developed separately from the interactive upload endpoint so that large-scale ingestion does not interfere with the responsiveness and reliability of user-facing analysis requests.

Reproducible model development: If fine-tuning is pursued, it should be implemented as a distinct training workflow using documented data splits, an appropriate learning objective, checkpoint selection, and a held-out test set. The fine-tuned model should be compared with the original pretrained model under the same evaluation conditions. Any reported improvement should be supported by measured results and an explicit description of the experimental setup.

These enhancements provide a practical development path. Establishing a reliable evaluation baseline should precede claims of improved accuracy, while persistent storage and batch-processing infrastructure should precede claims of catalogue-scale processing.

CHAPTER 8

8. CONCLUSION

The AI-Based Music Instrument Recognition System addresses the practical problem of analysing an uploaded music recording and presenting instrument-related evidence through an accessible web application. The project integrates audio-file validation, source separation, temporal segmentation, pretrained audio-text similarity matching, score aggregation, and frontend visualisation into a modular processing workflow. Its principal contribution is the integration of these components into an application that connects backend audio analysis with a user-facing interface.

The implementation uses FastAPI to handle requests and coordinate the backend workflow, Librosa to decode and prepare audio, Demucs to estimate source stems, and a pretrained CLAP model to compare short audio windows with text descriptions of instrument categories. The selected `other.wav` stem is analysed in one-second windows, with batched inference used to process the resulting segments. The backend aggregates the window-level evidence into song-level instrument match indices and retains temporal information that can be presented by the React frontend.

This design provides a useful separation of responsibilities. The backend manages audio processing and response generation, while the frontend manages file selection, preview, status feedback, and result presentation. The structured response also separates song-level summaries from window-level information, allowing the application to present an overall view alongside the variation of instrument-related evidence over time.

An important conclusion concerns the interpretation of the model output. The scores produced by the current implementation are similarity-derived indices rather than calibrated probabilities or measurements of each instrument's physical contribution to the original mix. Their usefulness depends on the relationship between the model's similarity estimates and actual instrument presence. This relationship must be established through labelled evaluation rather than assumed from the numerical scale of the output.

The project should therefore be assessed in terms of both implementation and experimental validation. Source inspection establishes the presence of the principal upload-to-analysis components and their intended interactions. It does not, by itself, demonstrate that all runtime scenarios have been tested successfully, that the model achieves a particular recognition accuracy, or that the estimated multi-million-file collection has been processed. These remain separate verification and evaluation requirements.

The current scope also distinguishes the implemented instrument-analysis workflow from the broader possibilities of Music Information Retrieval. Features such as genre and mood classification, tempo and key analysis, persistent catalogue similarity search, artist-style inference, and explanation generation require their own implementation and evaluation before they can be treated as verified capabilities. Similarly, model fine-tuning should be reported as completed only when a reproducible training procedure and measured comparison

with the pretrained baseline are available.

The next stage of development should prioritise reproducible runtime testing, verification of the dataset manifest, evaluation against labelled multi-instrument recordings, and transparent reporting of processing performance. A subsequent phase can investigate score calibration, alternative source-separation strategies, additional musical features, and scalable catalogue processing. Each extension should be introduced with a defined objective and evaluated against a documented baseline.

In conclusion, the project establishes a modular foundation for web-based instrument analysis using pretrained audio-text similarity. Its value lies in the integration of audio processing, temporal score generation, and interactive result presentation, together with a design that can support further experimentation. Establishing recognition accuracy, robustness, and large-scale processing capability remains dependent on systematic testing and measured evaluation. Maintaining this distinction between implemented functionality and experimentally verified performance provides a sound basis for further development of the system.

CHAPTER 9

9. REFERENCES

[1] R. Défossez, G. Synnaeve, and Y. Adi, "Real Time Speech Enhancement in the Waveform Domain," in Proc. Interspeech, 2020. (Demucs family of waveform-domain source-separation methods; consult the specific model version used for exact attribution.)

[2] Y. Wu et al., "Large-Scale Contrastive Language-Audio Pretraining with Feature Fusion and Keyword-to-Caption Augmentation," arXiv preprint arXiv:2211.06687, 2022.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[4] FastAPI, "FastAPI Documentation," https://fastapi.tiangolo.com/ (accessed October 2026).

[5] Librosa Development Team, "librosa: Audio and Music Signal Analysis in Python," https://librosa.org/doc/ (accessed October 2026).

[6] Hugging Face, "Transformers Documentation: CLAP," https://huggingface.co/docs/transformers/ (accessed October 2026).

[7] Meta AI, "Demucs: Music Source Separation," project repository and documentation, https://github.com/facebookresearch/demucs (accessed October 2026).

[8] React Team, "React Documentation," https://react.dev/ (accessed October 2026).

[9] Vite Team, "Vite Documentation," https://vite.dev/ (accessed October 2026).

Note: References are included for the principal technologies used or discussed. The final submission should retain only sources directly consulted and should verify the exact bibliographic details of each model and library citation.

APPENDIX A – SAMPLE CODE

A.1 FastAPI Analysis Endpoint (Simplified Extract)

```
@app.post("/api/analyze")
def analyze_song(file: UploadFile = File(...)):
extension = Path(file.filename or "upload").suffix.lower()
if extension not in {".mp3", ".wav", ".flac"}:
raise HTTPException(status_code=400, detail="Unsupported audio format")
# Save and validate the upload, run Demucs, then load CLAP.
# The complete implementation also validates size and audio decoding.
result = AnalysisService().analyze(str(other_wav))
return result
```

A.2 Score Aggregation Formula

For instrument i with window-level scores s(i,1), . . . , s(i,N), the current song-level match index is calculated as: MatchIndex(i) = 100 $\times$ (1/N) $\times$ $\sum$ s(i,n). Each score is first bounded to [0, 1]. Window coverage is calculated as 100 $\times$ (number of windows with score > 0 / N). These are the definitions implemented in AnalysisEngine.

A.3 API Response Fields

- analysis: per-window records containing timestamps and instrument scores.
- timelines: time-ordered score lists for each instrument category.
- song_level_scores: mean score per instrument, scaled to 0–100.
- contribution / instrument_contribution: compatibility aliases for song-level scores.
- window_coverage_percent / presence: percentage of windows with non-zero evidence.
- score_semantics: explanatory text clarifying that values are not probabilities or physical mix contributions.
- song: generated identifier, uploaded filename, and duration.